ScienceUtsav®
Science is awesome

I'm a ROBO SCIENTIST

Basics Of Light and Sound Circuits

Session 1

BASICS OF LIGHT AND SOUND CIRCUITS

Curiosity Kickoff

- Why do LEDs only glow in one direction (polarity)?
- Why does an LED need a resistor to work safely?
- What happens if you connect an LED directly to a battery without any control?

Concept Connection

This topic introduces students to Your First Arduino Project through the session Arduino Basics. The goal is to keep the learning simple, practical, and beginner-friendly. Students first understand the concept in easy words, then see a small Arduino program, and then connect the code to the real output or input behavior.

1. 1 Watt SMD LED

- I'm flat and powerful. Just one of me can light up your whole table like a torch.
- I don't like wasting energy, so I shine super bright without getting too hot.

1. 2 Buzzer

A buzzer is an electronic component that generates sound when electric current flows through it.

1. 3 Rocker Switch

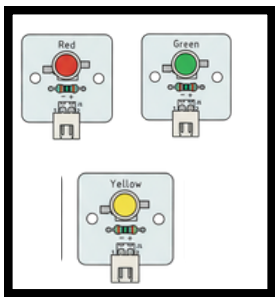
A rocker switch is an electrical device that allows you to turn a circuit ON or OFF by pressing one side of the switch.

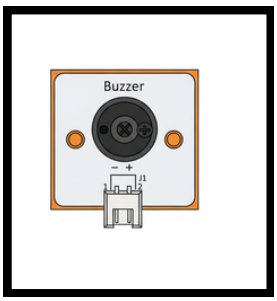
1. 4 Push Switch

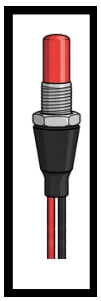
A push switch is a button-style switch that operates only when pressed.
It provides temporary input in circuits, such as starting a device or triggering an action.

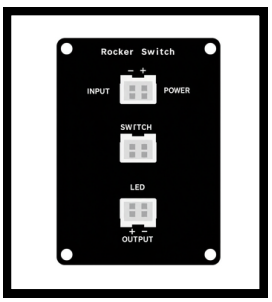
1. 5 Rocker switch PCB

A rocker switch PCB is a small circuit board designed to hold and connect a rocker switch easily in electronic projects.







Beginner: Blink LED with Delay 3 Patterns

Class Explanation

In this class, students are introduced to the Arduino board and a single LED. The focus is on making one LED blink so beginners feel comfortable with wiring a simple output and uploading their first sketch.

Materials Required

- Arduino with attached shield
- 1 × Red 1W SMD LED on breakout board
- RMC wires
- USB cable

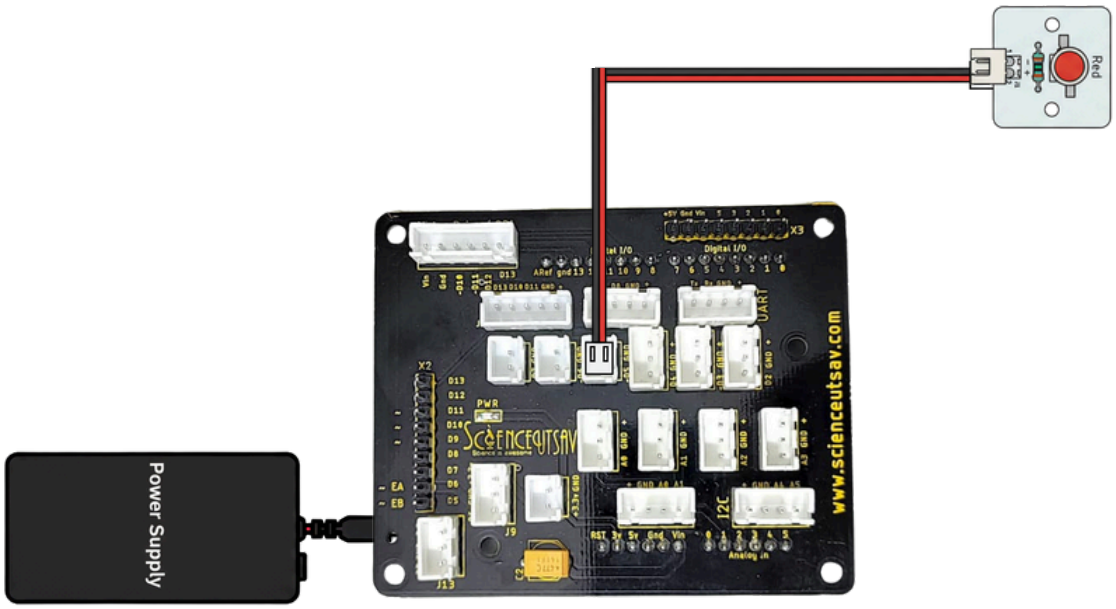
How to Connect

Use RMC wires to connect SMD LED breakout board directly to the Arduino:

- Red SMD LED → Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int ledPin = 5;

void setup() \
 pinMode(ledPin, OUTPUT);
\

void loop() \
 digitalWrite(ledPin, HIGH);
 delay(500);
 digitalWrite(ledPin, LOW);
 delay(500);
\
```

Code Explanation

- In an Arduino program, first we define the pins or variables. This means we give a name to a pin number so it is easy to use in the code. For example, if we write int ledPin = 5;, it means we are using pin number 5 and calling it ledPin.

- Next comes the setup() function. This part runs only one time when the Arduino is turned on. It is used to prepare the pins and start things like the Serial Monitor. Inside setup, we use pinMode() to tell the Arduino whether a pin will work as an input or an output. For example, setting a pin as OUTPUT allows it to control devices like an LED.

- After that comes the loop() function. This part runs again and again forever, so the Arduino keeps checking or controlling the circuit continuously.

- Inside the loop, we use digitalWrite() to turn an output ON or OFF. For example, digitalWrite(ledPin, HIGH); will turn the LED ON, and digitalWrite(ledPin, LOW); will turn it OFF.

So overall, the program first prepares everything in setup(), then keeps running the loop continuously, controlling devices like LEDs using functions such as pinMode() and digitalWrite().

Try This Yourself Code

This Try This Yourself code keeps the same topic of Introduction to Arduino and Components but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int ledPin = 5;

void setup() \
 pinMode(ledPin, OUTPUT);
\

void loop() \
 digitalWrite(ledPin, HIGH);
 delay(200);
 digitalWrite(ledPin, LOW);
 delay(200);
 digitalWrite(ledPin, HIGH);
 delay(800);
 digitalWrite(ledPin, LOW);
 delay(800);
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

- what Arduino is
- how to connect one LED as an output
- how to upload and test a very basic blink sketch

Intermediate: Outputs Inputs Overview

Class Explanation

In this class, students still work only with LEDs. The focus is on understanding that Arduino can control the with rocker switch to turn ON and OFF the LED.

Materials Required

- Arduino with attached shield
- Rocker switch with PCB
- 1 × Green 1W SMD LED on breakout board
- RMC wires
- USB cable

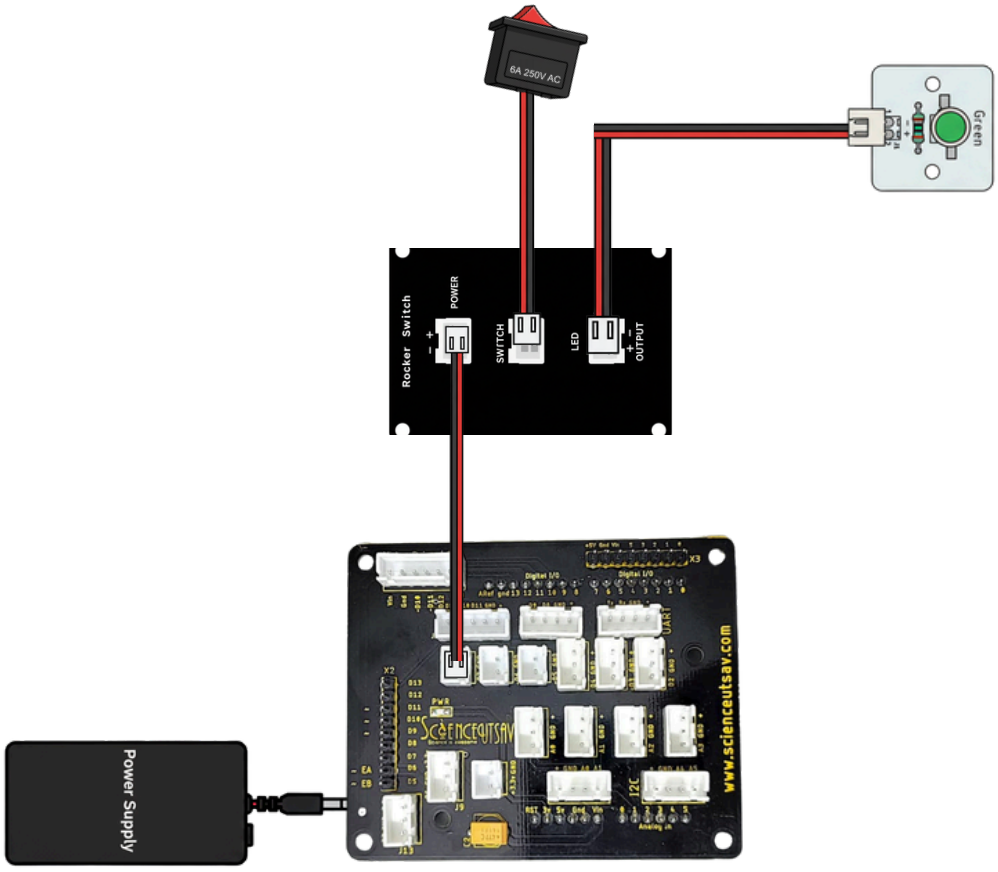
How to Connect

Use RMC wires to connect SMD LED breakout board directly to the Arduino:

- Rocker switch with PCB
- Green SMD LED → Arduino pin D6 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int led = 7;

void setup() {
 pinMode(led, OUTPUT);
}

void loop() {
 digitalWrite(led, HIGH);
}
```

Code Explanation

- In this Arduino program, a rocker switch is used to control an LED connected to pin 7 (led).
- The setup() function runs once when the Arduino starts. It sets the LED pin as an OUTPUT using pinMode().
- The loop() function runs continuously. It uses digitalWrite() to keep the LED pin HIGH, which keeps the LED turned ON.
- When the rocker switch is connected in series with the LED's power supply, turning the switch ON allows current to flow and the LED glows continuously. Turning the switch OFF cuts the power, and the LED turns OFF.

So overall, the rocker switch acts as a manual ON/OFF control for the LED.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Outputs Inputs Overview but changes the pins, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int led = 5;

void setup() {
 pinMode(led, OUTPUT);
}

void loop() {
 digitalWrite(led, HIGH);
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

- how Arduino controls one LED
- how output pins can work independently
- how changing the pins of arduino

Advanced: Pin Usage Basics

Class Explanation

In this class, students use multiple LEDs to understand that different Arduino pins can control different outputs. The idea is to build confidence with pin numbers, sequence, and simple wiring.

Materials Required

- Arduino with attached shield
- 1 × Red 1W SMD LED on breakout board
- 1 × Green 1W SMD LED on breakout board
- 1 × Yellow 1W SMD LED on breakout board
- RMC wires
- USB cable

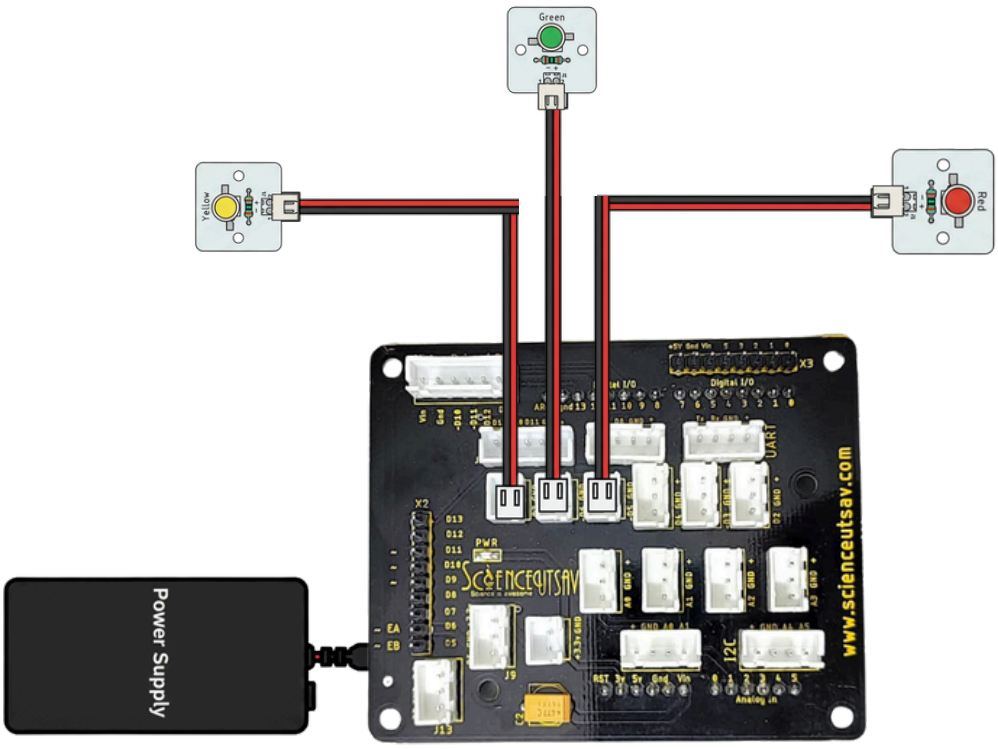
How to Connect

Use RMC wires to connect SMD LED breakout board directly to the Arduino:

- Red SMD LED → Arduino pin D5 and GND
- Green SMD LED → Arduino pin D6 and GND
- Yellow SMD LED → Arduino pin D7 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
const int led1 = 5;
const int led2 = 6;
const int led3 = 7;

void setup() \
 pinMode(led1, OUTPUT);
 pinMode(led2, OUTPUT);
 pinMode(led3, OUTPUT);
\

void loop() \
 digitalWrite(led1, HIGH);
 digitalWrite(led2, LOW);
 digitalWrite(led3, LOW);
 delay(300);

 digitalWrite(led1, LOW);
 digitalWrite(led2, HIGH);
 digitalWrite(led3, LOW);
 delay(300);

 digitalWrite(led1, LOW);
 digitalWrite(led2, LOW);
 digitalWrite(led3, HIGH);
 delay(300);
\
```

Code Explanation

- In an Arduino program, first we define the pins or variables to give names to pin numbers. The setup() function runs one time when the Arduino starts and is used to prepare the pins. Inside it, pinMode() tells whether a pin is input or output.

- After that comes the loop() function. This part runs again and again forever, so the Arduino keeps checking or controlling the circuit continuously.

So overall, setup() prepares the program once, and loop() keeps controlling the circuit continuously.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Pin Usage Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
const int led1 = 5;
const int led2 = 6;
const int led3 = 7;

void setup() \
 pinMode(led1, OUTPUT);
 pinMode(led2, OUTPUT);
 pinMode(led3, OUTPUT);
\

void loop() \
 digitalWrite(led1, HIGH);
 digitalWrite(led2, HIGH);
 digitalWrite(led3, LOW);
 delay(250);

 digitalWrite(led1, LOW);
 digitalWrite(led2, HIGH);
 digitalWrite(led3, HIGH);
 delay(250);

 digitalWrite(led1, HIGH);
 digitalWrite(led2, LOW);
 digitalWrite(led3, HIGH);
 delay(250);

 digitalWrite(led1, LOW);
 digitalWrite(led2, LOW);
 digitalWrite(led3, LOW);
 delay(250);
\
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

- how to use multiple LED output pins
- how to control different LEDs separately
- why pin numbers and sequence matter in Arduino code

Expert: System Design Thinking with Arduino

Class Explanation

In this class, students combine push button and buzzer. The aim is to think of the circuit as one simple alarm system where outputs work together in a clear sequence.

Materials Required

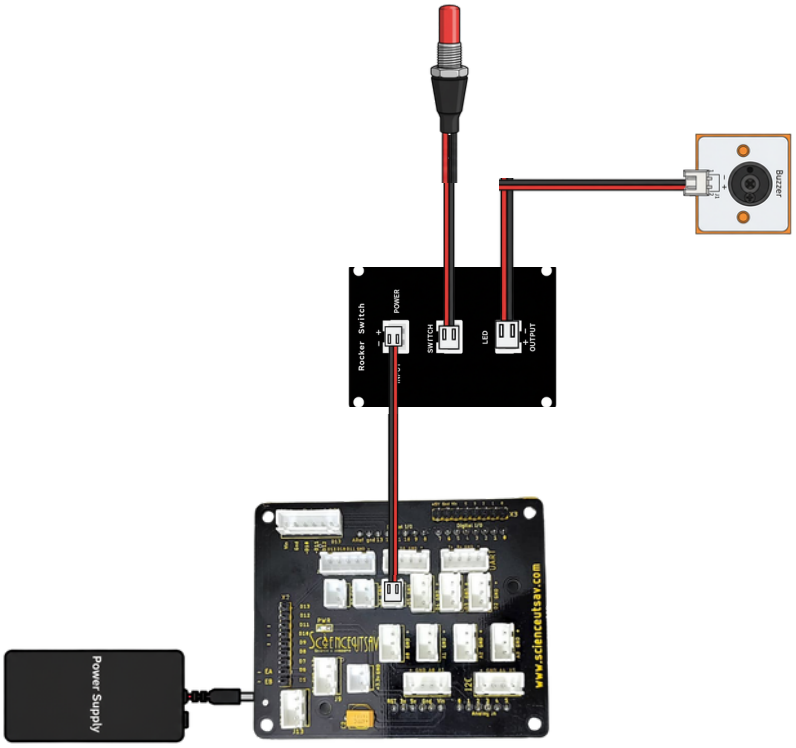
- Arduino with attached shield
- Rocker switch PCB
- 1 × Buzzer
- Push button
- RMC wires
- USB cable

How to Connect

- Rocker switch PCB → Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
/*
Simple Alarm Sound
Buzzer connected to pin 5
*/

const int buzzer = 5;

void setup() {
 pinMode(buzzer, OUTPUT);
}

void loop() {
 digitalWrite(buzzer,HIGH);
}
```

Code Explanation

- In this Arduino program, a push button is connected to pin 2 (buttonPin) and a buzzer is connected to pin 5 (buzzer).
- The setup() function runs once when the Arduino starts. It sets the push button as INPUT_PULLUP and the buzzer as OUTPUT using pinMode().
- The loop() function runs continuously. It reads the button state using digitalRead().
- If the push button is pressed (LOW signal), the buzzer turns ON using digitalWrite(HIGH). If the button is released (HIGH), the buzzer turns OFF using digitalWrite(LOW).
- So overall, the buzzer sounds only while the push button is pressed.

Try This Yourself Code

This Try This Yourself code keeps the same topic of System Design Thinking with Arduino but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
/*
Simple Alarm Sound
Buzzer connected to pin 7
*/

const int buzzer = 7;

void setup() {
 pinMode(buzzer, OUTPUT);
}

void loop() {
 digitalWrite(buzzer,HIGH);
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

- how a group of LEDs can act like one small system
- how to build a simple LED pattern using 1 to 3 outputs
- how to control LED by switch and push button

Learning Outcomes

After completing this session, students should be able to:

- what Arduino is
- how to connect one LED as an output
- how to upload and test a very basic blink sketch
- how Arduino controls two LEDs one after another
- how output pins can work independently
- how changing the sequence changes the visible pattern
- how to use multiple LED output pins
- how to control different LEDs separately
- why pin numbers and sequence matter in Arduino code
- how a group of LEDs can act like one small system
- how to build a simple LED pattern using 1 to 3 outputs
- how to think about order, timing, and overall behavior

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself codecode
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output